

DAY 1: Introduction to optimization using Linear Programming, Integer Programming and Constraint Programming

University of Ioannina, Dept. of Informatics and Telecommunications

https://github.com/chgogos/INTRO_OR_AI

INTRODUCTION TO OPTIMIZATION

- Descriptive analytics (Statistics)
- Predictive analytics (A.I.=Artificial Intelligence)
- Prescriptive analytics (O.R.=Operations Research)

O.R. and A.I. are complementary. Predictive models can inform prescriptive models, and that combination is where the real power lies



<https://doi.org/10.1287/orms.2022.04.14>

MATHEMATICAL PROGRAMMING

- Problem description → Mathematical Modeling → Mathematical Programming → Solve using solver (e.g., Gurobi, IBM ILOG CPLEX, Google OR-Tools, HIGHS, ...) → Evaluate results → Reiterate
- Mathematical Programming is a branch of O.R. (Operations Research)

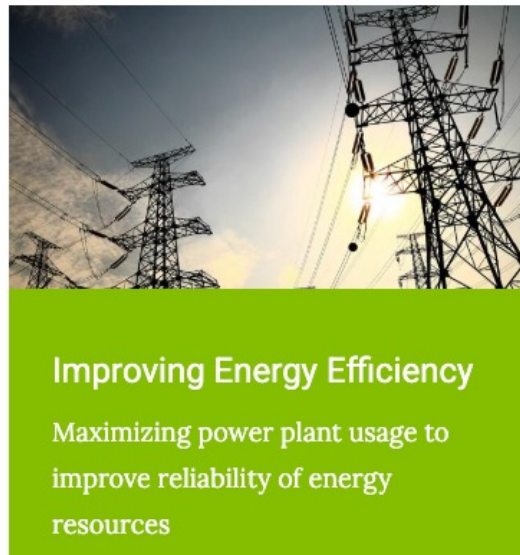
State of Mathematical Optimization Report, 2021

<https://cdn.gurobi.com/wp-content/uploads/2022/08/Report-StateOfMathematicalOptimizationReport.pdf>

85% of Fortune 500 companies use optimization

O.R. & ANALYTICS SUCCESS STORIES

INFORMS = Institute for Operations Research and the Management Sciences



<https://www.informs.org/Impact/O.R.-Analytics-Success-Stories>



Faster and Further
Improving train travel mile by mile
[Read More](#)



Transforming Aviation
Security
Safeguarding aircraft, crew and
passengers
[Read More](#)



Increasing Bike-share
Efficiency
Transforming how we travel
[Read More](#)



Enhancing Product
Platforms
Barco improves product features
and shortens production time
[Read More](#)



More Effective Television
Advertising
Turner Broadcasting uses analytics
to provide targeted advertising to
viewers
[Read More](#)



Improving Organ Donation
Pairing donors with patients in
need to save lives
[Read more](#)

A SIMPLE EXAMPLE OF MODELING: INVESTMENT OPPORTUNITIES

An amount of 2,000,000 € will be allocated across four investment opportunities, labelled A, B, C, and D. A minimum of 200,000 € and a maximum of 1,000,000 € must be invested in each opportunity. The combined investment in A and B must not exceed 1,500,000 €. The projected returns are 9%, 11%, 5%, and 8% for opportunities A, B, C, and D, respectively.

$$\text{Maximize } Z = 0.09x_A + 0.11x_B + 0.05x_C + 0.08x_D$$

s.t.

$$x_A + x_B + x_C + x_D = 2,000,000 \quad (1)$$

$$x_A \geq 200,000 \quad (2)$$

$$x_B \geq 200,000 \quad (3)$$

$$x_C \geq 200,000 \quad (4)$$

$$x_D \geq 200,000 \quad (5)$$

$$x_A \leq 1,000,000 \quad (6)$$

$$x_B \leq 1,000,000 \quad (7)$$

$$x_C \leq 1,000,000 \quad (8)$$

$$x_D \leq 1,000,000 \quad (9)$$

$$x_A + x_B \leq 1,500,000 \quad (10)$$

*Formal description
of the problem*

SOLUTION FOR THE SIMPLE INVESTMENT EXAMPLE USING OR-TOOLS

Maximize $Z = 0.09x_A + 0.11x_B + 0.05x_C + 0.08x_D$

s.t.

$$x_A + x_B + x_C + x_D = 2,000,000$$

$$x_A \geq 200,000$$

$$x_B \geq 200,000$$

$$x_C \geq 200,000$$

$$x_D \geq 200,000$$

$$x_A \leq 1,000,000$$

$$x_B \leq 1,000,000$$

$$x_C \leq 1,000,000$$

$$x_D \leq 1,000,000$$

$$x_A + x_B \leq 1,500,000$$

```
from ortools.linear_solver import pywraplp

solver = pywraplp.Solver.CreateSolver('SCIP')

# Decision variables
x_A = solver.NumVar(200_000, 1_000_000, 'x_A')
x_B = solver.NumVar(200_000, 1_000_000, 'x_B')
x_C = solver.NumVar(200_000, 1_000_000, 'x_C')
x_D = solver.NumVar(200_000, 1_000_000, 'x_D')

# Constraints
solver.Add(x_A + x_B + x_C + x_D == 2_000_000)

solver.Add(x_A + x_B <= 1_500_000)

# Objective
solver.Maximize(0.09 * x_A + 0.11 * x_B + 0.05 * x_C + 0.08 * x_D)

# Solve
status = solver.Solve()

if status == pywraplp.Solver.OPTIMAL:
    print(f'Optimal solution found:')
    print(f'x_A = {x_A.solution_value():.2f}')
    print(f'x_B = {x_B.solution_value():.2f}')
    print(f'x_C = {x_C.solution_value():.2f}')
    print(f'x_D = {x_D.solution_value():.2f}')
    print(f'Maximum Return = {solver.Objective().Value():.2f}')
else:
    print('The problem does not have an optimal solution.')
```

```
Optimal solution found:
x_A = 500,000.00
x_B = 1,000,000.00
x_C = 200,000.00
x_D = 300,000.00
Maximum Return = 189,000.00
```

PYTHON BASICS AND BEYOND!

- Lists, sets, tuples, dictionaries
 - Iterations over lists & dictionaries, enumerate
 - Functions
 - Assertions
 - OOP (classes, methods)
 - Comprehensions
 - f-strings
 - lambdas
 - Read data from text files, write data to text files
 - Measure execution time of code
 - Debugging
- <https://learnxinyminutes.com/python/>
 - Python As Fast as Possible - Learn Python in ~75 Minutes -
<https://www.youtube.com/watch?v=VchuKL44s6E>
 - <https://marko-knoeb1.github.io/slides/python-all-en.html>
 - <https://programming-25.mooc.fi/>

PACKAGE MANAGERS + VIRTUAL ENVIRONMENTS

- UV: modern high performance Python package manager (10-100x faster than pip)
- UV can act as a replacement for the traditional tools pip and virtualenv (also for conda, poetry)
- UV provides better cross-platform consistency than its peers
- UV installation instructions: <https://docs.astral.sh/uv/getting-started/installation/>

```
$ uv init prj1
Initialized project `prj1` at `/Users/a_user/dev_git_repos/prj1`
$ cd prj1
$ tree -a
.
├── main.py
├── pyproject.toml
├── README.md
└── uv.lock
$ uv add ortools
Using CPython 3.11.11
Creating virtual environment at: .venv
Resolved 11 packages in 682ms
Prepared 3 packages in 6.94s
Installed 10 packages in 84ms
...
+ ortools==9.12.4544
...
```

```
main.py
1  import sys
2  import ortools
3
4  # Display Python version
5  print(f"Python version: {sys.version}")
6
7  # Display OR-Tools version
8  print(f"OR-Tools version: {ortools.__version__}")
9
```

```
$ uv run main.py
Python version: 3.11.11 (main, Dec 19 2024,
14:23:18) [Clang 18.1.8 ]
OR-Tools version: 9.12.4544
```

EXAMPLE: SEND + MORE = MONEY

```
1 from ortools.sat.python import cp_model
2
3 model = cp_model.CpModel()
4
5 s = model.new_int_var(1, 9, "S") # cannot be 0
6 e = model.new_int_var(0, 9, "E")
7 n = model.new_int_var(0, 9, "N")
8 d = model.new_int_var(0, 9, "D")
9 m = model.new_int_var(1, 9, "M") # cannot be 0
10 o = model.new_int_var(0, 9, "O")
11 r = model.new_int_var(0, 9, "R")
12 y = model.new_int_var(0, 9, "Y")
13
14 letters = [s, e, n, d, m, o, r, y]
15
16 model.add_all_different(letters)
17
18 # SEND + MORE = MONEY
19 send = 1000 * s + 100 * e + 10 * n + d
20 more = 1000 * m + 100 * o + 10 * r + e
21 money = 10000 * m + 1000 * o + 100 * n + 10 * e + y
22
23 model.add(send + more == money)
24
25 solver = cp_model.CpSolver()
26 status = solver.solve(model)
27
28 if status == cp_model.FEASIBLE or status == cp_model.OPTIMAL:
29     print(f"S = {solver.value(s)}")
30     print(f"E = {solver.value(e)}")
31     print(f"N = {solver.value(n)}")
32     print(f"D = {solver.value(d)}")
33     print(f"M = {solver.value(m)}")
34     print(f"O = {solver.value(o)}")
35     print(f"R = {solver.value(r)}")
36     print(f"Y = {solver.value(y)}")
37     print()
38     print(f"{solver.value(send)} + {solver.value(more)} = {solver.value(money)}")
```

```
$ uv run cryptarithmic.py
S = 9
E = 5
N = 6
D = 7
M = 1
O = 0
R = 8
Y = 2

9567 + 1085 = 10652
```

```
SEND
+ MORE
= MONEY

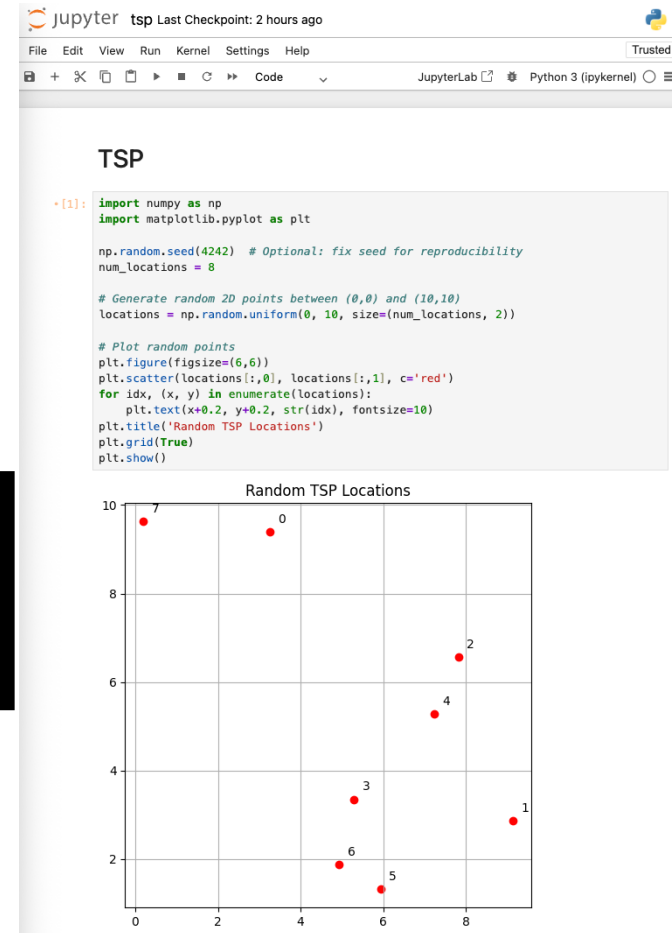
9567
+ 1085
= 10652
```

How about:
TWO + TWO = FOUR
?

JUPYTER NOTEBOOK

- Jupyter notebook is a popular tool for interactive computing, data analysis, and visualization

```
$ uv init prj2
Initialized project `prj2` at `~/Users/a_user/dev_git_repos/prj2`
$ cd prj2
$ uv add jupyter ortools matplotlib numpy
...
$ uv run jupyter notebook
[I 2025-04-28 11:50:20.242 ServerApp] http://localhost:8888/tree?token=c96782dbcf41a97121a3bb5382da454b1a6d2375da26e0e7
[I 2025-04-28 11:50:20.242 ServerApp] http://127.0.0.1:8888/tree?token=c96782dbcf41a97121a3bb5382da454b1a6d2375da26e0e7
[I 2025-04-28 11:50:20.242 ServerApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
...
```



VSCODE + PYTHON

- <https://marko-knoebl.github.io/slides/python-beginner-collection-en.html> - VS Code for Python

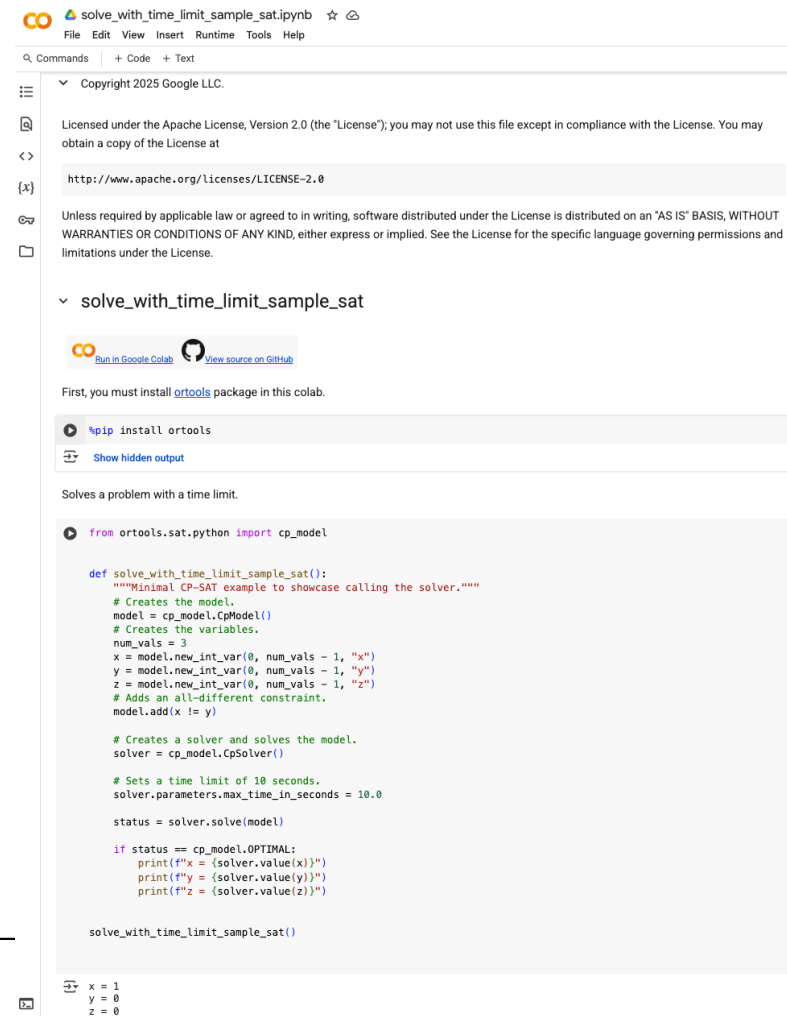


GOOGLE COLAB

- Google's Colab is a cloud based Jupyter Notebook service that requires no setup to use and provides **free access to computing resources**
- Google colab notebooks can be saved into Google Drive



https://colab.research.google.com/github/google/ortools/blob/stable/examples/notebook/sat/solve_with_time_limit_sample_sat.ipynb



```
def solve_with_time_limit_sample_sat():
    """Minimal CP-SAT example to showcase calling the solver."""
    # Creates the model.
    model = cp_model.CpModel()
    # Creates the variables.
    num_vals = 3
    x = model.new_int_var(0, num_vals - 1, "x")
    y = model.new_int_var(0, num_vals - 1, "y")
    z = model.new_int_var(0, num_vals - 1, "z")
    # Adds an all-different constraint.
    model.add(x != y)

    # Creates a solver and solves the model.
    solver = cp_model.CpSolver()

    # Sets a time limit of 10 seconds.
    solver.parameters.max_time_in_seconds = 10.0

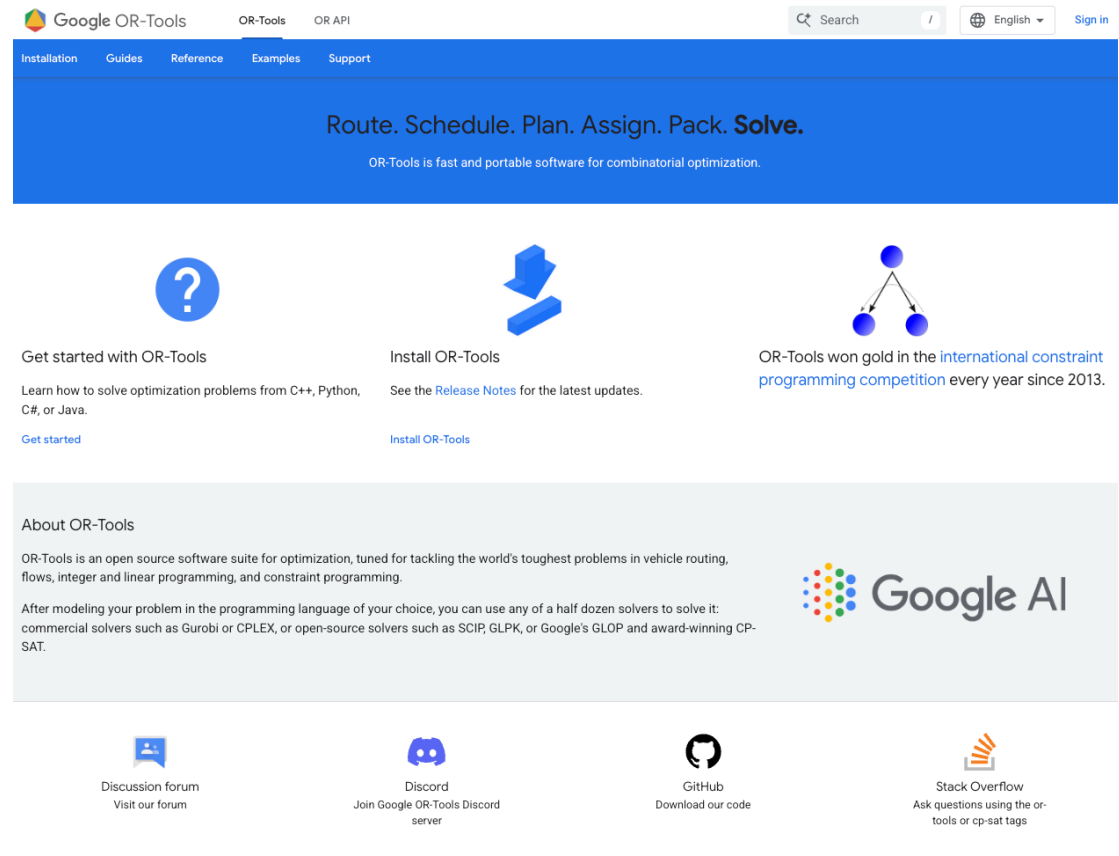
    status = solver.solve(model)

    if status == cp_model.OPTIMAL:
        print(f"x = {solver.value(x)}")
        print(f"y = {solver.value(y)}")
        print(f"z = {solver.value(z)}")

solve_with_time_limit_sample_sat()
```

GOOGLE OR-TOOLS - RESOURCES

- <https://developers.google.com/optimization>
- https://github.com/or-tools/awesome_or_tools
- <https://d-krupke.github.io/cpsat-primer/>
- <https://kunlei.github.io/python-ortools-notes/>



The screenshot shows the Google OR-Tools website. At the top, there's a navigation bar with links for Installation, Guides, Reference, Examples, and Support. The main header features the text "Route. Schedule. Plan. Assign. Pack. **Solve.**" and a sub-header stating "OR-Tools is fast and portable software for combinatorial optimization." Below this, there are three main sections: "Get started with OR-Tools" (with a question mark icon), "Install OR-Tools" (with a download icon), and "OR-Tools won gold in the international constraint programming competition every year since 2013." (with a tree diagram icon). The "About OR-Tools" section describes it as an open source software suite for optimization, tuned for tackling the world's toughest problems in vehicle routing, flows, integer and linear programming, and constraint programming. It also mentions that after modeling a problem in a programming language of choice, users can use any of a half dozen solvers to solve it, including commercial solvers like Gurobi or CPLEX, or open-source solvers like SCIP, GLPK, or Google's GLOP and award-winning CP-SAT. At the bottom, there are four social media links: Discussion forum, Discord, GitHub, and Stack Overflow.

Google OR-Tools OR-Tools OR API Search English Sign in

Installation Guides Reference Examples Support

Route. Schedule. Plan. Assign. Pack. **Solve.**

OR-Tools is fast and portable software for combinatorial optimization.

Get started with OR-Tools

Learn how to solve optimization problems from C++, Python, C#, or Java.

[Get started](#)

Install OR-Tools

See the [Release Notes](#) for the latest updates.

[Install OR-Tools](#)

OR-Tools won gold in the [international constraint programming competition](#) every year since 2013.

About OR-Tools

OR-Tools is an open source software suite for optimization, tuned for tackling the world's toughest problems in vehicle routing, flows, integer and linear programming, and constraint programming.

After modeling your problem in the programming language of your choice, you can use any of a half dozen solvers to solve it: commercial solvers such as Gurobi or CPLEX, or open-source solvers such as SCIP, GLPK, or Google's GLOP and award-winning CP-SAT.

Google AI

Discussion forum
Visit our forum

Discord
Join Google OR-Tools Discord server

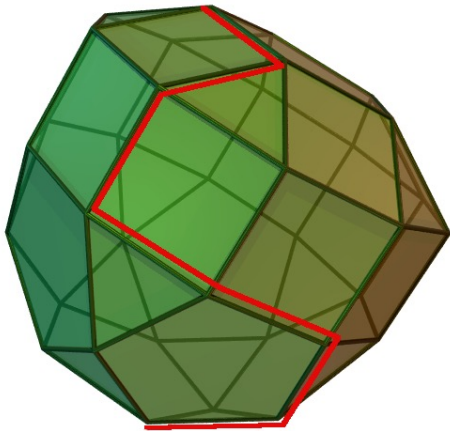
GitHub
Download our code

Stack Overflow
Ask questions using the or-tools or cp-sat tags

INTRODUCTION TO OPTIMIZATION USING LINEAR PROGRAMMING, INTEGER PROGRAMMING AND CONSTRAINT PROGRAMMING

Model and solve decision support problems using Google's OR-Tools and Python

LINEAR PROGRAMMING



- Linear Programming (LP) is used a lot in practice
- LP entails formulation (modeling) and then solving of linear programs
- Basic elements of a Linear Program:
 - **Decision variables** (assume continuous values)
 - **Constraints** (functions should be linear over decision variables)
 - **Objective function** (either maximize or minimize of a linear function over the decision variables)
- The general formulation of a mathematical program:

$$\begin{array}{ll}\max & f(x_1, x_2, \dots, x_n) \\ \text{s. t.} & g_i(x_1, x_2, \dots, x_n) \leq b_i \quad \forall i = 1, \dots, m \\ & x_j \in \mathbb{R} \quad \forall j = 1, \dots, n\end{array}$$

- Concrete example of a problem instance:

$$\begin{array}{ll}\max & x_1 + x_2 \\ \text{s. t.} & x_1 + 2x_2 \leq 6 \\ & 2x_1 + x_2 \leq 6 \\ & x_1 \geq 0, x_2 \geq 0\end{array}$$

LINEAR PROGRAMMING – COMPACT LP FORMULATIONS

- Most problem instances, in practice, have big (or huge!) numbers of variables and constraints
- Compact formulations enhance readability by use of **indices** (e.g., i, j, k), the **summation** symbol (Σ) and the **for all** symbol ($\forall$)
- Example of compact formulation:

$$\begin{array}{ll}\min & \sum_{j \in J} c_j x_j \\ \text{s.t.} & \sum_{j \in J} a_{ij} x_j \geq b_i, \quad \forall i \in I \\ & x_j \geq 0, \quad \forall j \in J\end{array}$$

where x_j are the decision variables for each $j \in J$,
 c_j is the cost coefficient for each $j \in J$,
 a_{ij} is the coefficient of variable j in constraint i ,
 b_i is the right-hand side value for constraint i ,
 I, J are finite index sets.

LINEAR PROGRAMMING – ALGORITHMS & SOLVERS

- Effective algorithms exist for solving LPs:
 - Simplex method (Dantzig, 1947) and its variants, i.e., Revised Simplex, Dual Simplex, Primal-Dual Simplex
 - Interior point methods (Karmarkar, 1984), i.e., Barrier
 - ~~Ellipsoid method (Khachiyan, 1979)~~
 - First-order methods (for large LPs), i.e., Primal-Dual Hybrid Gradient
- Commercial and **open-source** software provide state of the art (SoTA) implementations of LP solvers:
 - Gurobi
 - IBM ILOG CPLEX
 - FICO XPRESS
 - MOSEK
 - **OR-Tools (GLOP, PDLP, and 3rd party LP solvers)**
 - **HIGHS (LP solvers)**
 - **SCIP Optimization Suite (SoPlex)**
 - **COIN-OR (CLP)**
 - **GLPK**

- The relative performance of algorithms and solvers can vary by orders of magnitude on a single instance.
- No single algorithm and solver is uniformly better than others.
- Crossover (enabled by default) increases solve time, sometimes substantially.
- PDLP can solve to low precision sometimes 10 times faster than to high precision.

https://developers.google.com/optimization/lp/lp_advanced

LP EXAMPLE – DIET PROBLEM (PROBLEM STATEMENT)

You are tasked with planning a daily diet using 5 available food items. The goal is to minimize the total cost while ensuring that the diet provides at least the required daily amounts of 3 essential nutrients: **Calories**, **Protein**, and **Vitamins**. Each food item has a known cost per unit and provides specific amounts of these nutrients. The **nutritional requirements** are at least 1,200 calories per day, at least 50 grams of protein per day and at least 5 grams of vitamins per day.

Food	Cost per Unit (€)	Calories (per unit)	Protein (grams per unit)	Vitamins (grams per unit)
Food 1	2	300	5	1
Food 2	1.5	200	10	2
Food 3	3	400	8	3
Food 4	2	250	6	2
Food 5	4	500	12	4

Nutritional Information for Each Food Item

DIET PROBLEM (FORMULATION X 2)

Food	Cost per Unit (€)	Calories (per unit)	Protein (grams per unit)	Vitamins (grams per unit)
Food 1	2	300	5	1
Food 2	1.5	200	10	2
Food 3	3	400	8	3
Food 4	2	250	6	2
Food 5	4	500	12	4

Formulation

$$\begin{aligned} \min \quad & Z = 2x_1 + 1.5x_2 + 3x_3 + 2x_4 + 4x_5 \\ \text{s.t.} \quad & 300x_1 + 200x_2 + 400x_3 + 250x_4 + 500x_5 \geq 1200 \\ & 5x_1 + 10x_2 + 8x_3 + 6x_4 + 12x_5 \geq 50 \\ & x_1 + 2x_2 + 3x_3 + 2x_4 + 4x_5 \geq 5 \\ & x_1, x_2, x_3, x_4, x_5 \geq 0 \end{aligned}$$

Alternative formulation (compact)

$$\min \quad Z = \mathbf{c}^T \mathbf{x}$$

$$\text{s.t.} \quad A\mathbf{x} \geq \mathbf{b}$$

$$\mathbf{x} \geq 0$$

where

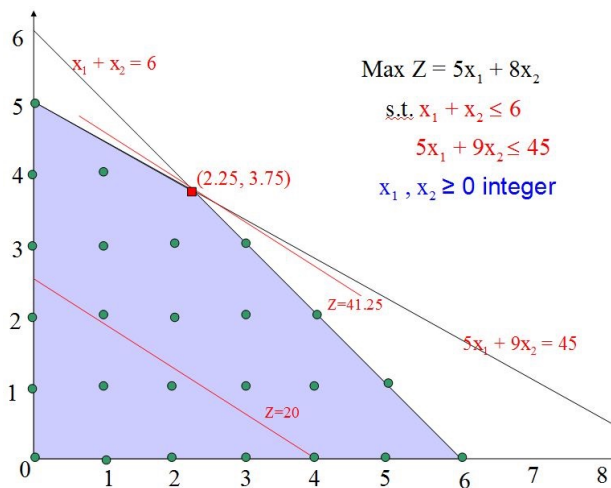
$$\mathbf{c} = \begin{pmatrix} 2 \\ 1.5 \\ 3 \\ 2 \\ 4 \end{pmatrix}, \quad A = \begin{pmatrix} 300 & 200 & 400 & 250 & 500 \\ 5 & 10 & 8 & 6 & 12 \\ 1 & 2 & 3 & 2 & 4 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 1200 \\ 50 \\ 5 \end{pmatrix}$$

DIET PROBLEM (SOLUTION X 2 USING OR-TOOLS GLOP)

```
1 from ortools.linear_solver import pywraplp
2
3 # Create the solver
4 solver = pywraplp.Solver.CreateSolver('GLOP')
5
6 # Variables (continuous)
7 x1 = solver.NumVar(0.0, solver.infinity(), 'x1')
8 x2 = solver.NumVar(0.0, solver.infinity(), 'x2')
9 x3 = solver.NumVar(0.0, solver.infinity(), 'x3')
10 x4 = solver.NumVar(0.0, solver.infinity(), 'x4')
11 x5 = solver.NumVar(0.0, solver.infinity(), 'x5')
12
13 # Objective function
14 solver.Minimize(2 * x1 + 1.5 * x2 + 3 * x3 + 2 * x4 + 4 * x5)
15
16 # Constraints
17 solver.Add(300 * x1 + 200 * x2 + 400 * x3 + 250 * x4 + 500 * x5 >= 1200) # Calories
18 solver.Add(5 * x1 + 10 * x2 + 8 * x3 + 6 * x4 + 12 * x5 >= 50) # Protein
19 solver.Add(x1 + 2 * x2 + 3 * x3 + 2 * x4 + 4 * x5 >= 5) # Vitamins
20
21 # Solve the problem
22 status = solver.Solve()
23
24 # Results
25 solution = {}
26 if status == pywraplp.Solver.OPTIMAL:
27     solution = {
28         'x1': x1.solution_value(),
29         'x2': x2.solution_value(),
30         'x3': x3.solution_value(),
31         'x4': x4.solution_value(),
32         'x5': x5.solution_value(),
33         'objective': solver.Objective().Value()
34     }
35
36 print(solution)
```

```
1 from ortools.linear_solver import pywraplp
2
3 # Create the solver
4 solver = pywraplp.Solver.CreateSolver('GLOP')
5
6 # Data (costs, nutritional values)
7 c = [2, 1.5, 3, 2, 4] # Cost per unit for each food
8 A = [
9     [300, 200, 400, 250, 500], # Calories
10    [5, 10, 8, 6, 12],          # Protein
11    [1, 2, 3, 2, 4]             # Vitamins
12 ]
13 b = [1200, 50, 5] # Minimum nutritional requirements
14
15 # Variables (continuous)
16 x = [solver.NumVar(0.0, solver.infinity(), f'x{i+1}') for i in range(5)]
17
18 # Objective function: Minimize c^T * x
19 objective = solver.Objective()
20 for i in range(5):
21     objective.SetCoefficient(x[i], c[i])
22 objective.SetMinimization()
23
24 # Constraints: A * x >= b
25 for i in range(3): # 3 constraints
26     constraint = solver.Constraint(b[i], solver.infinity())
27     for j in range(5): # 5 foods
28         constraint.SetCoefficient(x[j], A[i][j])
29
30 # Solve the problem
31 status = solver.Solve()
32
33 # Results
34 solution = {}
35 if status == pywraplp.Solver.OPTIMAL:
36     solution = {
37         'x': [x[i].solution_value() for i in range(5)],
38         'objective': solver.Objective().Value()
39     }
40
41 print(solution)
```

INTEGER PROGRAMMING



- In some problems, variables must only assume **integer values**
- **Integer Programming (IP)** is the process of formulating and solving problems with integer variables
- An IP is actually an Integer Linear Program (ILP)
- **Mixed Integer Programming (MIP)** and **Binary Integer Programming (BIP)** are special cases of IPs
- **Non-Linear Programs (NLPs)** are another class of problems (e.g., using the quadratic function over a decision variable in the objective or the constraints)

INTEGER PROGRAMMING – ALGORITHMS & SOLVERS

- Effective* algorithms exist for solving IPs:
 - Branch and Bound (B&B) – solves linear relaxations and systematically partitions the problem space (branching) while using bounds (bounding) to prune subproblems
 - Branch and Cut (B&C) – an extension to B&B that adds cutting planes (valid inequalities) during the search to tighten the LP relaxation and speed up convergence
- Commercial and **open-source** software provide SoTA implementations of IP solvers:
 - Gurobi
 - IBM ILOG CPLEX
 - FICO XPRESS
 - **OR-Tools (uses 3rd party IP solvers such as SCIP and CBC)**
 - **HIGHS (MIP solver)**
 - **SCIP Optimization Suite (SCIP)**
 - **COIN-OR (CBC)**
 - **GLPK (supports both LP and IP)**

*although, less effective than their LP counterparts

IP EXAMPLE: THE 0/1 KNAPSACK PROBLEM

The 0/1 Knapsack Problem is a classic combinatorial optimization problem where, given a set of items each with a weight and a value, the goal is to determine the most valuable subset of items to include in a knapsack without exceeding its weight capacity



$$\begin{aligned} \max \quad & \sum_{i=1}^n v_i x_i \\ \text{s.t.} \quad & \sum_{i=1}^n w_i x_i \leq W \\ & x_i \in \{0, 1\}, \quad i = 1, 2, \dots, n \end{aligned}$$

where:

- v_i = value (or profit) of item i ,
- w_i = weight of item i ,
- W = capacity of the knapsack.

There are special purpose algorithms for the knapsack problem that can solve it efficiently (e.g., Dynamic Programming)

THE 0/1 KNAPSACK PROBLEM – FORMULATION

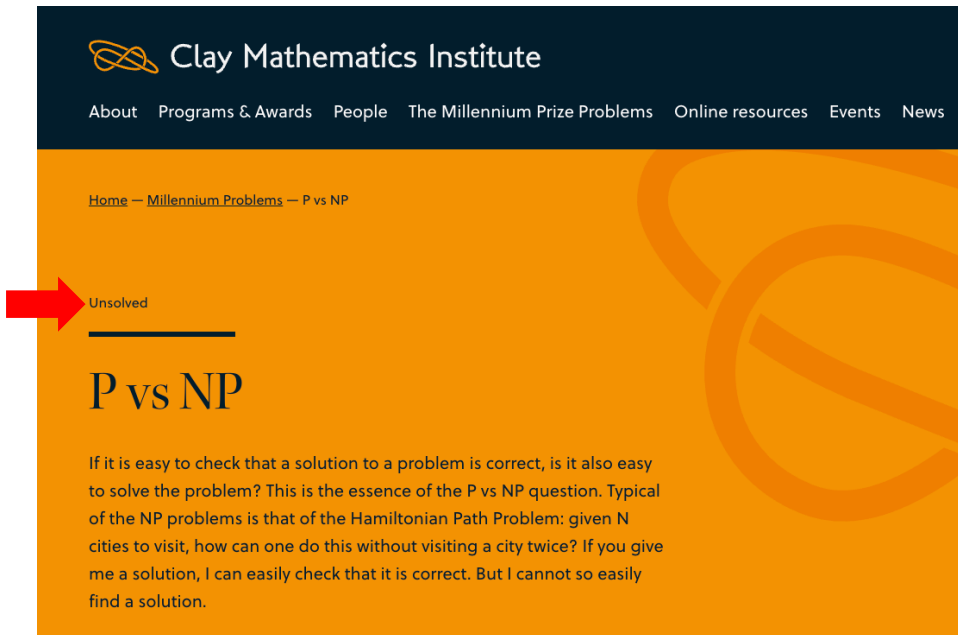


Item	1	2	3	4	5
Value (\$)	4	2	1	2	10
Weight (kg)	12	1	1	2	4

$$\begin{aligned} \max \quad & 4x_1 + 2x_2 + x_3 + 2x_4 + 10x_5 \\ \text{s.t.} \quad & 12x_1 + x_2 + x_3 + 2x_4 + 4x_5 \leq 15 \\ & x_i \in \{0, 1\}, \quad \forall i = 1, \dots, 5 \end{aligned}$$

```
1  from ortools.linear_solver import pywraplp
2
3  solver = pywraplp.Solver.CreateSolver("SCIP")
4
5  # Binary Decision Variables
6  x = []
7  for i in range(5):
8      x.append(solver.IntVar(0, 1, f"x{i+1}"))
9
10 # Objective function
11 solver.Maximize(4 * x[0] + 2 * x[1] + x[2] + 2 * x[3] + 10 * x[4])
12
13 # Constraint
14 solver.Add(12 * x[0] + x[1] + x[2] + 2 * x[3] + 4 * x[4] <= 15)
15
16 status = solver.Solve()
17
18 if status == pywraplp.Solver.OPTIMAL:
19     print("Solution:")
20     for i in range(5):
21         print(f"x{i+1} = {int(x[i].solution_value())}")
22     print(f"Maximum objective value = {solver.Objective().Value()}")
23 else:
24     print("No optimal solution found.")
```

P VS. NP PROBLEM



Clay Mathematics Institute

About Programs & Awards People The Millennium Prize Problems Online resources Events News

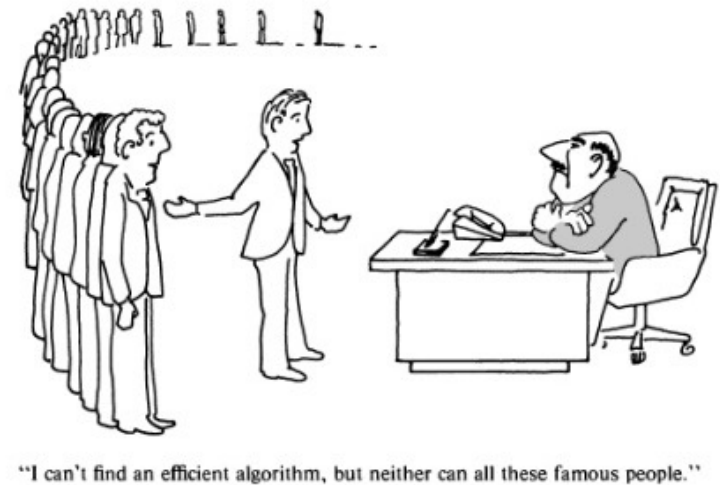
Home — Millennium Problems — P vs NP

Unsolved

P vs NP

If it is easy to check that a solution to a problem is correct, is it also easy to solve the problem? This is the essence of the P vs NP question. Typical of the NP problems is that of the Hamiltonian Path Problem: given N cities to visit, how can one do this without visiting a city twice? If you give me a solution, I can easily check that it is correct. But I cannot so easily find a solution.

<https://www.claymath.org/millennium/p-vs-np/>



An example of an NP problem is the **Subset Sum Problem (SSP)**: Given a set of integers and a target sum T , is there a subset whose sum equals T ?

SSP is directly related to the 0/1 knapsack problem

CONSTRAINT PROGRAMMING

“Constraint programming represents one of the closest approaches computer science has yet made to the Holy Grail of programming: the user states the problem, the computer solves it.”

Eugene Freuder, 1996

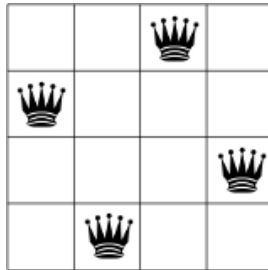
- **Constraint Programming (CP)** is a paradigm for solving combinatorial problems by stating constraints that must be satisfied
- Core idea: The problem is modelled using variables that have finite domains (e.g., $x \in \{1, 2, 3, 4\}$, $y \in \{1 \dots 10\}$, $z = \{2, 4, 6, 8, 10\}$) and constraints (e.g., $3x \neq y$, $x * y = z$)
- The solving process:
 - Constraint propagation (reduce domains by enforcing constraints)
 - Search (explore assignments using backtracking if necessary)
- CP can be used for feasibility problems and for optimization problems
- CP enables a more flexible modelling framework than LP and IP, as it supports general combinatorial and logical constraints beyond linear forms

CONSTRAINT PROGRAMMING – ALGORITHMS & SOLVERS

- Several algorithms have been developed for solving Constraint Programming problems:
 - Arc consistency (e.g., AC-3, AC-4, AC-2001)
 - Global constraint propagators (e.g., all-different, if-then, cumulative)
 - Backtracking search and Backjumping
 - Branch and Bound (B&B)
 - Heuristics (e.g., MRV, LCV)
 - Lazy Clause Generation (LCG)
 - Large Neighborhood Search (LNS)
 - Commercial and **open-source** software provide SoTA implementations of CP solvers:
 - IBM ILOG CPLEX (CP-Optimizer)
 - FICO XPRESS (XPRESS-CP)
 - **OR-Tools (CP, CP-SAT)**
 - **Choco Solver**
 - **Gecode**
 - **Chuffed**
-

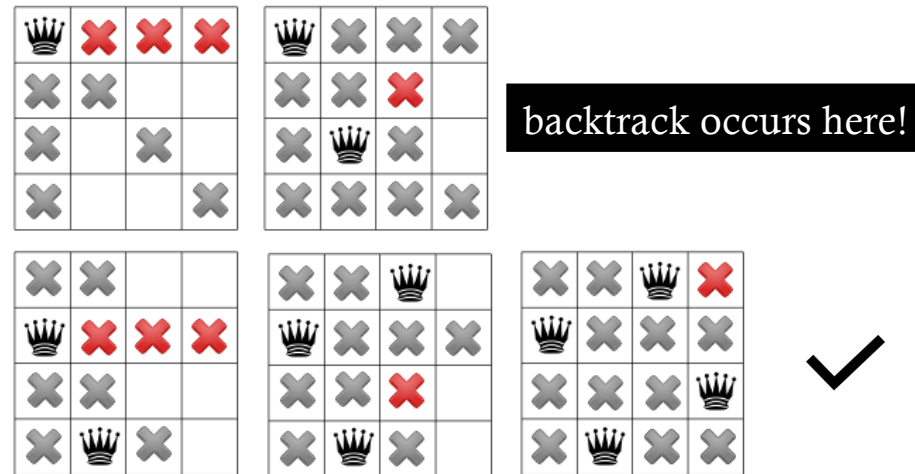
CP EXAMPLE – THE N-QUEENS PROBLEM

How can N queens be placed on an NxN chessboard so that no two of them attack each other?



<https://developers.google.com/optimization/cp/queens>

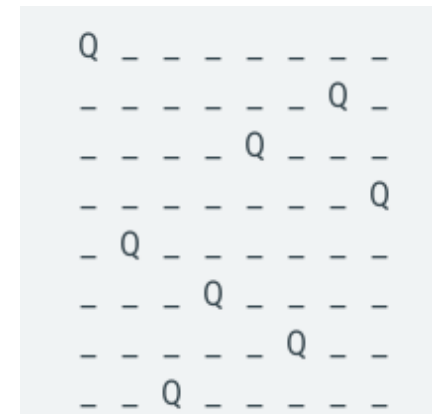
- N-Queens is not an optimization problem, it is a feasibility problem
- Propagation + backtracking



THE N-QUEENS PROBLEM – FORMULATION

```
1 from ortools.constraint_solver import pywrapcp
2
3 def main(board_size):
4     solver = pywrapcp.Solver("n-queens")
5
6     # Variables: The array index is the column, and the value is the row.
7     queens = [solver.IntVar(0, board_size - 1, f"x{i}") for i in range(board_size)]
8
9     # All rows must be different.
10    solver.Add(solver.AllDifferent(queens))
11
12    # No two queens can be on the same diagonal.
13    solver.Add(solver.AllDifferent([queens[i] + i for i in range(board_size)]))
14    solver.Add(solver.AllDifferent([queens[i] - i for i in range(board_size)]))
15
16    db = solver.Phase(queens, solver.CHOOSE_FIRST_UNBOUND, solver.ASSIGN_MIN_VALUE)
17
18    solver.NewSearch(db)
19    while solver.NextSolution():
20        for i in range(board_size):
21            for j in range(board_size):
22                if queens[j].Value() == i:
23                    print("Q", end=" ")
24                else:
25                    print("_", end=" ")
26            print()
27        print()
28    solver.EndSearch()
29
30 main(8)
```

1 of the 92 solutions for the 8 x 8 board



queens[0]=0
queens[1]=4
...
queens[7]=3

A different formulation, that solves the problem using IP, can be found at:
<https://kunlei.github.io/python-ortools-notes/ip-nqueens.html>

THE N-QUEENS PROBLEM – IMPLEMENTATION USING CP-SAT

```
1 from ortools.sat.python import cp_model
2
3
4 def solve(board_size):
5     # Create the solver
6     model = cp_model.CpModel()
7
8     # Variables: The array index is the column, and the value is the row.
9     queens = [model.new_int_var(0, board_size - 1, f"x{i}") for i in range(board_size)]
10
11     # All rows must be different.
12     model.add_all_different(queens)
13
14     # No two queens can be on the same diagonal.
15     model.add_all_different([queens[i] + i for i in range(board_size)])
16     model.add_all_different([queens[i] - i for i in range(board_size)])
17
18     # Solve the model.
19     solver = cp_model.CpSolver()
20     solver.solve(model)
21
22     # Display the first solution found and terminate.
23     all_queens = range(board_size)
24     for i in all_queens:
25         for j in all_queens:
26             if solver.value(queens[j]) == i:
27                 # There is a queen in column j, row i.
28                 print("Q", end=" ")
29             else:
30                 print("_", end=" ")
31         print()
32
33
34 if __name__ == "__main__":
35     solve(8)
```

The code on the left displays only the first solution

Q	_	_	_	_	_	_	_
_	_	_	_	_	_	Q	_
_	_	_	Q	_	_	_	_
_	_	_	_	_	_	_	Q
_	Q	_	_	_	_	_	_
_	_	Q	_	_	_	_	_
_	_	_	_	Q	_	_	_
_	Q	_	_	_	_	_	_

HANDS-ON ACTIVITIES



Resource Allocation



Workforce Planning



Cryptarithmic puzzle

HANDS-ON – RESOURCE ALLOCATION – THE DIET PROBLEM

1. Find what the “Stigler Diet” is by consulting either the Web or Open AI’s ChatGPT or Antropic’s Claude or Deepseek or any other Large Language Model (LLM)
2. Navigate through the OR-Tools web site and **locate the code** that using LP finds the optimal solution for the “Stigler diet”
3. Execute locally (or in Google Collab) the code, using GLOP (i.e., the Linear Programming solver developed as part of OR-Tools) and another LP solver (e.g., CLP of COIN-OR) and compare the results
4. What would have been the cost of the optimal solution in current money value (you can use <https://data.bls.gov/cgi-bin/cpicalc.pl>) ?

Extra

- Visit: <https://www.feedcalculator.org/>
- Read: Dantzig, George B. “The diet problem.” Interfaces 20.4 (1990): 43-47.



HANDS-ON – WORKFORCE PLANNING

- Assume that we operate a department store and we know the number of employees needed for each day of the week. Each employee must work for five consecutive days. How many employees should we hire for each one of the 7 shifts:
 - Monday to Friday = ?
 - Tuesday to Saturday = ?
 - Wednesday to Sunday = ?
 - Thursday to Monday = ?
 - Friday to Tuesday = ?
 - Saturday to Wednesday = ?
 - Sunday to Thursday = ?
- The goal is to minimize the number of employees hired

Mon	Tue	Wed	Thu	Fri	Sat	Sun
90	80	100	60	80	150	140

Tip: Use 7 decision variables, the first one should assume the value of the number of employees that are hired in shift Monday to Friday, the second one should assume the value of the number of employees that are hired in shift Tuesday to Saturday, etc.

HANDS-ON – WORKFORCE PLANNING – FORMULATION

$$\begin{array}{ll}\min & x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 \\ \text{s.t.} & x_1 + x_4 + x_5 + x_6 + x_7 \geq 90 \\ & x_1 + x_2 + x_5 + x_6 + x_7 \geq 80 \\ & x_1 + x_2 + x_3 + x_6 + x_7 \geq 100 \\ & x_1 + x_2 + x_3 + x_4 + x_7 \geq 60 \\ & x_1 + x_2 + x_3 + x_4 + x_5 \geq 80 \\ & x_2 + x_3 + x_4 + x_5 + x_6 \geq 150 \\ & x_3 + x_4 + x_5 + x_6 + x_7 \geq 140 \\ & x_i \geq 0 \quad \forall i = 1, \dots, 7\end{array}$$

HANDS-ON – CRYPTARITHMETIC PUZZLES

Solve the following cryptarithmic puzzle:

$$\text{NINE} - \text{FOUR} = \text{FIVE}$$

The constraints are the following:

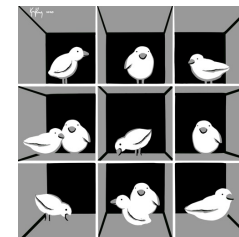
- All letters map to distinct digits
- Leading letters cannot be zero
- The subtraction must hold numerically

Use OR-Tools CP-SAT to find a single solution (if it exists)

Do you believe that the following cryptarithmic puzzles have solutions?

$$\text{BASE} \times \text{BALL} = \text{GAMES}$$

$$\text{PEACE} + \text{MIND} = \text{RELAX}$$



MANY OTHER OPTIMIZATION APPROACHES EXIST

- Gradient Descent (GD) & Stochastic Gradient Descent (SGD)
- Dynamic Programming (DP)
- Simulated Annealing (SA)
- Tabu Search (TS)
- Genetic Algorithms (GA)
- Particle Swarm Optimization (PSO)
- Ant Colony Optimization (ACO)
- Iterated Local Search (ILS)
- Variable Neighborhood Search (VNS)
- Random Key Optimizers (RKO)
- Reinforcement Learning (RL)
- Quantum Annealing (QA)
- Quantum Unconstrained Binary Optimization (QUBO)

NO FREE LUNCH (NFL) THEOREM

- There is no universally best optimization algorithm across all problems
- When the performance of all optimization methods is averaged across all conceivable problems, they all perform equally well



Wolpert, David H., and William G. Macready. "No free lunch theorems for optimization." IEEE transactions on evolutionary computation 1.1 (1997): 67-82.

REFERENCES

- The Burrito Optimization Game, <https://doi.org/10.1287/orms.2022.04.14>
- The state of the Mathematical Optimization Report 2021, <https://cdn.gurobi.com/wp-content/uploads/2022/08/Report-StateOfMathematicalOptimizationReport.pdf>
- CP-SAT Primer, <https://d-krupke.github.io/cpsat-primer/>
- ACP Summer School 2023: "Intro to Constraint Programming" by Tias Guns, <https://www.youtube.com/watch?v=JpxbiEAF80&list=PLcByDTr7vRTYJ2s6DL-3bzjGwtQif33y3>
- List of optimization packages in Python, <https://feloopy.github.io/blog/list-of-optimization-packages.html>
- Freuder, E. C. (1997). In pursuit of the holy grail. Constraints, 2, 57-61.
- Wolpert, David H., and William G. Macready. "No free lunch theorems for optimization." IEEE transactions on evolutionary computation 1.1 (1997): 67-82.